Hide Assignment Information

**Instructions**

# Recitation 06: Dynamic Arrays and Copy Control

## Topics

- **Copy Control** (aka Big 3)
- operator[]

## Attachments

- rec06_start.cpp
- rec06-output.txt
- rec06 FAQ.pdf
- A diagram depicting how the "pointer to pointer" is used in this problem
- rec06 Sample Code Reading Question

## Task

You will develop a class to represent a directory of employees.

### Starter code

We provide you with some code to start with in the attached file **rec06_start.cpp**. (Of course the file you will turn in will be named **rec06.cpp.**) The starter code provides the basic class definitions that we will need for modeling a directory of employees in a company. In particular, it provides the code you will need for the classes **Entry** and **Position**, along with a start for the Directory class.

Your job is to:

- Read and understand the given code.
    - Yes, the Entry class is nested and private. But that shouldn't affect you. We just figured, "why not?"
    - There is **no reason** to modify the Entry or Position classes. So **don't**.
- Obviously, the Directory class needs a constructor. We will make one constraint here, start the capacity at zero.

- Fill in the missing code for the Directory add method.
- Overload Directory's output operator. Test your code now!
- Overload Directory's [] operator to allow looking up a person's phone number, by passing in their name.
- **[Strongly recommend that you get checked out now!]**
- Implement the **copy control** functions (aka the Big 3), **yes all of them**!
    - **destructor**,
    - **copy constructor** and
    - **assignment operator**
    - *just* for the Directory class.
    - At the beginning of each of these functions, add a print statement to show when you have entered them. This will help you understand when / where they are each being used.
- You should (as always) consider if there is any way to further expand the code in main() to test all you your new features.

# Dynamic Array?

Some might ask, "Why are we using a dynamic array of Entry pointers for our Directory?" Sure, you are **[much]** more likely to use a vector or other container type (e.g. map), but this provides you with a good exercise in implementing *copy control*, which is the main point of this exercise.

Note that the Directory is ***responsible* for** both the Entries **and** the dynamic array itself. However the Position objects are on the stack, so are never deleted.

You should *think* about how using a vector of pointers would change your code. (No, you don't have to write anything, but do think.)

# Submit

A single file, **rec06.cpp**.

---

**Submit** **Cancel**